

Mastering sed Command in Linux: A Comprehensive Guide

Published on January 9, 2025

[Linux Commands](#)

[Linux Basics](#)



By [Anish Singh Walia](#)

Sr Technical Content Strategist and Team Lead



[Table of contents](#)



[Popular topics](#)



Introduction

The `sed` (Stream Editor) command in Linux is a powerful utility for text manipulation. It enables users to perform various operations such as searching, replacing, inserting, and deleting text in files. This tutorial provides a comprehensive guide to mastering the `sed` command with practical examples, syntax explanations, and advanced use cases.

What is the sed Command?

The `sed` command is a [stream editor](#) that processes text in a line-by-line fashion. This allows you to modify file content without directly opening the file in a text editor. It is widely used in shell scripting and system administration to automate text-processing tasks.

Key Features of `sed`

1. Pattern matching and replacement 2. In-place file editing 3. Text filtering and manipulation 4. Support for regular expressions 5. Multiline operations

Basic Syntax of the `sed` Command

The basic syntax of the `sed` command consists of three main components: the command options, a script defining the editing instructions, and the file to be processed.

This structure allows users to specify the command's behavior, define the text transformations, and apply them to the desired file.

- 1. Command Options:** These are used to specify the command's behavior. For example, the `-i` option is used for in-place file editing i.e to overwrite the file.
- 2. Script:** The script defines the editing instructions. It can be enclosed in single quotes (`'`) or double quotes (`"`). The script can contain one or more editing commands, each separated by a semicolon (`;`).
- 3. Input File:** This is the file to be processed. It can be a single file or a list of files separated by spaces. If no file is specified, `sed` reads from the standard input.

The basic syntax of the `sed` command is as follows:

```
sed [options] 'script' file
```

In this syntax, `sed` is the command, `[options]` is the command options, `'script'` contains the editing commands, and `file` is the file to be processed.

```
sed [options] 'script' file
```

You will understand it better with the examples below.

```
sed 's/hello/world/' sample.txt
```

This replaces the first occurrence of "hello" with "world" in each line of `sample.txt`.

Commonly Used Options in sed

Option	Description	Example
-i	In-place editing	<code>sed -i 's/old/new/' file.txt</code>
-n	Suppress automatic printing	<code>sed -n '/pattern/p' file.txt</code>
-e	Execute multiple commands	<code>sed -e 's/old/new/' -e '/pattern/d' file.txt</code>
-f	Read commands from a file	<code>sed -f script.sed file.txt</code>
-r	Use extended regular expressions	<code>sed -r 's/old/new/' file.txt</code>
-E	Use extended regular expressions (similar to -r)	<code>sed -E 's/old/new/' file.txt</code>

-z	Separate lines with NUL character	<code>sed -z 's/old/new/' file.txt</code>
-l	Specify the line length for the 'l' command	<code>sed -l 100 'l' file.txt</code>
-b	Binary mode (do not strip the CR characters)	<code>sed -b 's/old/new/' file.txt</code>

Most Common Use Cases of sed

Below are some of the most practical use cases of the `sed` command.

First, let's create a sample text file `file1.txt` and write the below text to it, for ease of understanding and follow along-

```
cat > file1.txt
```

Copy-paste the following text:

```
Linux is a family of free and open-source operating systems based on the Linux kernel. Operating systems based on Linux are known as Linux distributions or distributions. Examples include Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Linux, and many others.
```

Search and Replace

In the below command the `s` specifies the substitution operation and the `/` are delimiters. The `/Linux/` is the search pattern and the `Unix` is the replacement string.

Note: By default, the `sed` command only replaces the first occurrence of the pattern in each line and it won't replace the second or third occurrences in the line.

```
sed 's/Linux/Unix/' file1.txt
```

This command replaces the first occurrence of “Linux” with “Unix” in each line.

Output:

Output

```
Unix is a family of free and open-source operating systems based on the L
Operating systems based on Unix are known as Linux distributions or distr
Examples include Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Unix , and m
```

Replace Globally in Each Line

The substitute flag `/g` (global replacement) specifies the `sed` command to replace all the occurrences of the string in the line.

```
sed 's/Linux/Unix/g' file1.txt
```

This command replaces all occurrences of “Linux” with “Unix” in each line.

Output:

Output

Unix is a family of **free** and open-source operating systems based on the Linux kernel. Operating systems based on Unix are known as Unix distributions or distros. Examples include Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Linux, and many others.

In-Place Editing

The `-i` enables in-place editing of the file. In simple words it overwrites the file.

```
sed -i 's/Linux/Unix/' file1.txt
```

This command edits the file in place, replacing "Linux" with "Unix" directly in `file1.txt`. Without `-i`, the insertion occurs only in the output and doesn't modify the file content. To make the change persistent, you need to use the `-i` option.

Delete Specific Lines

```
sed '2d' file1.txt
```

This command will delete the second line from `file1.txt`.

Output:

Output

Unix is a family of **free** and open-source operating systems based on the Linux kernel. Examples include Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Linux, and many others.

Print Specific Lines

The `-n` suppresses automatic printing of pattern space and `p` is the print command.

```
sed -n '1,2p' file1.txt
```

This command prints lines 1 through 2 from `file1.txt` .

Output:

Output

Unix is a family of `free` and open-source operating systems based on the Un.
Operating systems based on Unix are known as Unix distributions or distros

Delete Lines Matching a Pattern

The `/pattern/` matches lines containing the pattern and the `d` flag deletes matched lines.

```
sed '/kernel/d' file1.txt
```

This command will delete all the lines containing the word "kernel".

Output:

Output

Operating systems based on Unix are known as Unix distributions or distros
Examples include Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Unix, and mai

Substitute with a Backup File

The below command will replace all the instances of "Unix" with "Linux" and create a backup file named `file1.txt.bak` with older file content before replacing. The `-i.bak` enables in-place editing and creates a backup file.

```
sed -i.bak 's/Unix/Linux/g' file1.txt
```

Output:

Output

```
Linux is a family of free and open-source operating systems based on the
Operating systems based on Linux are known as Linux distributions or dist
Examples include Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Linux , and
```

And, here is the file content of the backup file `file1.txt.bak` .

```
more file1.txt.bak
```

Output

```
Unix is a family of free and open-source operating systems based on the Un
Operating systems based on Unix are known as Unix distributions or distros
Examples include Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Unix, and ma
```

Replace Tabs with Spaces

The below command will replace each tab character with four spaces. The `\t` flag matches tab characters and the `/g` flag is for the global replacement across the line.


```
sed 's/\t/ /g' file1.txt
```

Output:

Output

Linux is a family of **free** and open-source operating systems based on the
Operating systems based on Linux are known as Linux distributions or distro
Examples include Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Linux, and

Remove Empty Lines

The below command will delete all the empty lines from `file1.txt`. The `/^$` matches empty lines and `/d` flag deletes matched lines.

```
sed '/^$/d' file1.txt
```

You can edit the `file1.txt` file using `vi` text editor and add some empty lines to test this command.

Print Lines Matching a Pattern

The below command prints only the lines containing "Ubuntu".

```
sed -n '/Ubuntu/p' file1.txt
```

The `-n` option suppresses automatic printing. `/Ubuntu/` matches lines containing

the pattern. and the `p` prints matched lines.

Output:

Output

Example includes Debian, Ubuntu , Fedora, CentOS, Gentoo, Arch Linux, and

Advanced Use Cases of sed

This section consists of some advanced and more complicated use cases of the `sed` command.

Insert Text Before a Line

The below command inserts "This is inserted text." before the second line in `file1.txt`.

```
sed -i '2i\This is inserted text.' file1.txt
```

The `-i` option is for in-place editing and the `2i\` flag inserts text before the 2nd line.

Note: Without `-i`, the insertion occurs only in the output and doesn't modify the file content. To make the change persistent, you need to use the `-i` option with the `sed` command.

Output:

Output

```
Linux is a family of free and open-source operating systems based on the L
This is inserted text.
Operating systems based on Linux are known as Linux distributions or distr
Example includes Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Linux, and m
```

Replacing the nth occurrence of a pattern in a line

Use the `/1` or `/2` flags to replace the first, second occurrence of a pattern in a line. The below command replaces the second occurrence of the word "Linux" with "Unix" in a line.

```
sed 's/Linux/Unix/2' file1.txt
```

Output:

Output

```
Linux is a family of free and open-source operating systems based on the L
This is inserted text.
Operating systems based on Linux are known as Unix distributions or distr
Examples includes Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Linux, and r
```

Append String After a Line

The below command appends "This is appended text." after the third line in `file1.txt`. The `-i` option makes sure the changes are saved and the `3a\` appends text after the specified third line.

```
sed -i '3a\This is appended text.' file1.txt
```

Output:

Output

```
Linux is a family of free and open-source operating systems based on the L
This is inserted text.
```

```
Operating systems based on Linux are known as Linux distributions or distro
This is appended text.
```

```
Example includes Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Linux, and m
```

Replace String at the Beginning of a Line

The `^<pattern>` flag is used to match a specific pattern at the start of a line. The below command replaces "Linux" with "Unix" only if "Linux" appears at the start of a line.

```
sed 's/^Linux/Unix/' file1.txt
```

Output:

Output

```
Unix is a family of free and open-source operating systems based on the L
This is inserted text.
```

```
Operating systems based on Linux are known as Linux distributions or distro
This is appended text.
```

```
Example includes Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Linux, and m
```

Replace String at the End of a Line

The below command replaces "distros." with "distributions" only if it appears at the end of a line. The `<pattern>$` flag is used to match a specific pattern to the end of a line.

```
sed 's/distros.$/distributions/' file1.txt
```

Output:

Output

```
Linux is a family of free and open-source operating systems based on the L
This is inserted text.
Operating systems based on Linux are known as Linux distributions or distr
This is appended text.
Example includes Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Linux, and ma
```

Case-Insensitive Replacement

The below command replaces "linux" with "Unix" ignoring case sensitivity. The `I` flag makes the match case-insensitive.

```
sed 's/linux/Unix/I' file1.txt
```

Extract Lines Between Patterns

The below command prints all lines between "inserted" and "appended", inclusive.

```
sed -n '/inserted/,/appended/p' file1.txt
```

1. `, :` Range operator to match lines between two patterns.
2. `p` : Prints matched lines.

And the `-n` option to suppress automatic printing of lines.

Output:

Output

```
This is inserted text.
```

```
Operating systems based on Linux are known as Linux distributions or distribu
```

```
This is appended text.
```

Process Multiple Files

The following command replaces "Linux" with "Unix" in both `file1.txt` and `file2.txt` and overwrites the file.

```
sed -i 's/Linux/Unix/' file1.txt file2.txt
```

Format and Number Non-Empty Lines

The below command adds line numbers to non-empty lines in `file1.txt`.

```
sed '/./=' file1.txt | sed 'N;s/\n/ /'
```

1. `/./=` : Matches non-empty lines and numbers them.
2. `N` : Appends the next line to the pattern space.
3. `s/\n/ /` : Replaces the newline character with a space.

Output:

Output

```
1 Linux is a family of free and open-source operating systems based on the
2 This is inserted text.
3 Operating systems based on Linux are known as Linux distributions or dis
4 This is appended text.
5 Example includes Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Linux, and
```

Replacing string on a specific line number

You can restrict the `sed` command to replace the string on a specific line number. The below command replaces the string "distros" with "distributions" only on the third line.

```
sed '3 s/distros/distributions/' file1.txt
```

Output:

```
Linux is a family of free and open-source operating systems based on the L
This is inserted text.
Operating systems based on Linux are known as Linux distributions or distr
This is appended text.
Example includes Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Linux, and m
```

Replacing string on a range of lines

You can also specify a range of line numbers to the `sed` command for replacing a string. The below command replaces only the first occurrences of "Linux" with "Unix" between lines 1 to 3.

```
sed '1,3 s/Linux/Unix/' file1.txt
```

Output:

Output

```
Unix is a family of free and open-source operating systems based on the L
This is inserted text.
Operating systems based on Unix are known as Linux distributions or distr
This is appended text.
Example includes Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Linux, and m
```

Performance Considerations for Large Files

Processing large files with `sed` can become resource-intensive, especially when dealing with numerous operations or very large datasets. Here are some tips to optimize performance and ensure efficient use of the `sed` command:

1. Use `-n` to minimize unnecessary output - The `-n` option suppresses automatic printing of each line and ensures only the desired output is displayed. This reduces overhead when working with large files.

Example:

```
sed -n '/pattern/p' largefile.txt
```

2. Simplify Scripts - Minimize the number of operations in a single command. For instance, instead of applying multiple `sed` commands sequentially, combine them into a single script to reduce file reads.

Example:


```
sed -e 's/foo/bar/' -e '/pattern/d' largefile.txt
```

3.Stream Input with Pipes: - When processing data from other commands or streams, use pipes to avoid intermediate file creation and reduce disk I/O.

Example:

```
cat largefile.txt | sed 's/foo/bar/' > output.txt
```

4.Avoid In-Place Editing on Large Files - Instead of directly modifying large files, write output to a new file and replace the original after verifying correctness.

Example:

```
sed 's/old/new/' largefile.txt > temp.txt && mv temp.txt largefile.txt
```

5.Benchmark Alternatives - For very large files, consider using tools like `awk` , `perl` , or `grep` , which may offer better performance for certain tasks.

Example:

```
awk '{gsub(/old/, "new"); print}' largefile.txt > output.txt
```

You can refer to these tutorials on [AWK command in Linux](#) and [How To Use the AWK language to Manipulate Text in Linux](#) to learn more about using `awk` command in linux.

Integration with Shell Scripts

The `sed` command is commonly used in [shell scripts](#) to automate repetitive text manipulation tasks. Here's an example:

```
#!/bin/bash
# Replace all occurrences of "foo" with "bar" in input.txt and save the result
sed 's/foo/bar/g' input.txt > output.txt
```

This script processes `input.txt` and writes the modified output to `output.txt`.

sed vs Other Alternatives

While `sed` is an effective and lightweight tool for basic text processing, modern alternatives such as `awk` and `perl` offer additional functionality, making them better suited for specific tasks. Here's a breakdown of key differences and when to use each:

When to Use `sed`

- Quick, simple text substitutions or deletions.
- Line-based transformations in files.
- Tasks requiring minimal scripting overhead.

When to Use `awk`

- Handling structured data such as CSV or TSV files.
- Performing arithmetic calculations alongside text processing.
- Generating formatted reports from input data.

Example:

```
awk -F, '{print $1, $3}' data.csv
```

This extracts and prints the first and third fields from a CSV file.

When to Use `perl`

- Complex text manipulations involving advanced regular expressions.
- Combining text processing with logic or conditions.
- Writing compact, yet powerful scripts.

Example:

```
perl -pe 's/(error)/WARNING: $1/' logfile.txt
```

This adds a "WARNING:" prefix to lines containing the word "error".

Conclusion

Mastering the `sed` command enhances your ability to manipulate and process text efficiently in Linux. Its powerful features and seamless integration into scripts make it a valuable tool for text-based manipulation tasks.

Next Steps

After mastering the basics of `sed`, you can learn more advanced techniques and use cases. You can use the below series of tutorials on `sed` and related topics that can help you deepen your understanding and improve your text processing skills:

- [The Basics of Using the `sed` Stream Editor to Manipulate Text in Linux](#)
- [Intermediate `sed` : Manipulating Streams of Text in a Linux Environment](#)

These tutorials cover various topics, from basic `sed` operations to more complex text manipulation techniques. They are a valuable resource for anyone looking to become proficient in text processing on the command line.

FAQs

What is sed command in Linux?

The `sed` (Stream Editor) command in Linux is a powerful text processing tool used to perform basic text transformations on an input stream (a file or input from a pipeline). It allows you to search, replace, delete, and insert text, making it highly useful for automating text manipulation tasks.

When to use sed ?

You can use `sed` in the following scenarios:

- **Text Replacement:** Replace words, phrases, or patterns in files or streams.
- **Text Deletion:** Remove specific lines or patterns.
- **In-place Editing:** Modify files directly without needing to open a text editor.
- **Batch Processing:** Perform the same operation on multiple files using scripts.
- **Text Insertion/Extraction:** Insert or extract specific text in structured files like configuration files or logs.

How to use sed properly?

To use `sed` effectively, follow these steps:

```
sed [options] 'command' file
```

- `command` : The `sed` operation (e.g., `s` for substitute, `d` for delete).
- `file` : The target file to process.

Test Before Applying In-place: First, run the command without the `-i` option to see the output before modifying files directly.

Use Regular Expressions: Leverage `sed`'s support for regular expressions to match and manipulate complex patterns.

Chain Multiple Commands: Use `;` or `-e` to execute multiple `sed` commands in a single operation.

How to use sed to replace text?

To replace text, use the substitute `s` command with this syntax:

```
sed 's/old_text/new_text/' file
```

Examples:

- Replace the first occurrence of "foo" with "bar" on each line:

```
sed 's/foo/bar/' file.txt
```

- Replace all occurrences of "foo" with "bar" globally:

```
sed 's/foo/bar/g' file.txt
```

- In-place replacement (modifies the file directly):

```
sed -i 's/foo/bar/g' file.txt
```

How do I run a sed command?

You can run a `sed` command directly from the terminal using this basic syntax:

```
sed 'command' filename
```

Example:

To print lines containing the word "error" and replace "error" with "warning" in a file named `log.txt` :

```
sed 's/error/warning/' log.txt
```

What is the difference between `grep` and `sed` commands in Linux?

Feature	grep	sed
Purpose	Search for patterns in one or more files	Edit streams of text
Output	Prints lines containing the pattern	Prints the edited text
Actions	Search, Filter	Search, Replace, Insert, Delete
Usage	grep pattern file	sed 'command' file
Search and Replace	Yes (limited)	Yes
In place Editing	No	Yes

In-place Editing	No	Yes
Regular Expressions	Yes	Yes
Multiline Operations	No	Yes
Text Filtering	No	Yes
Common Use Cases	Searching logs, Finding patterns in text	Editing configuration files, Replacing text in multiple files

Example:

- Using `grep` to search for "error" in `log.txt` :

```
grep 'error' log.txt
```

- Using `sed` to replace "error" with "warning" in `log.txt` :

```
sed 's/error/warning/g' log.txt
```

How to remove an empty line using `sed` ?

To remove empty lines from a file, use the following `sed` command:

```
sed '/^$/d' file.txt
```

Explanation:

- `^$` : Matches empty lines (lines with no characters).
- `d` : Deletes the matched lines.

Example:

Before running the command, a file might look like this:

```
line 1  
line 2  
line 3
```

After running the command:

```
sed '/^$/d' file.txt
```

The output will be:

```
Output  
line 1  
line 2  
line 3
```


Thanks for learning with the DigitalOcean Community. Check out our offerings for compute, storage, networking, and managed databases.

[Learn more about our products](#)

About the author



Anish Singh Walia

Author

Sr Technical Content Strategist and Team Lead

[See author profile](#)

I help Businesses scale with AI x SEO x (authentic) Content that revives traffic and keeps leads flowing | 3,000,000+ Average monthly readers on Medium | Sr Technical Writer(Team Lead) @ DigitalOcean | Ex-Cloud Consultant @ AMEX | Ex-Site Reliability Engineer(DevOps)@Nutanix